ESDC Meeting 0x4

Clocks and Timers

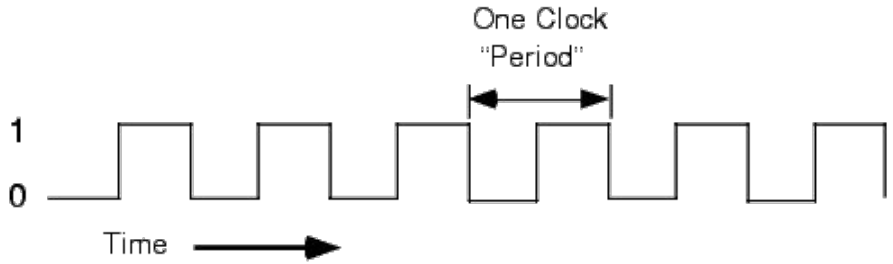
Agenda/Announcements

- We were originally going to go over PWM today

- Week 9 is thanksgiving
  - How would we feel about moving our next meeting to Week 8?

Why do I care about Clocks?

- Clocks are the heartbeat of our system
- Everything in our system is synced to our clock
  - Code execution
  - Moving Data
  - Configuring Register



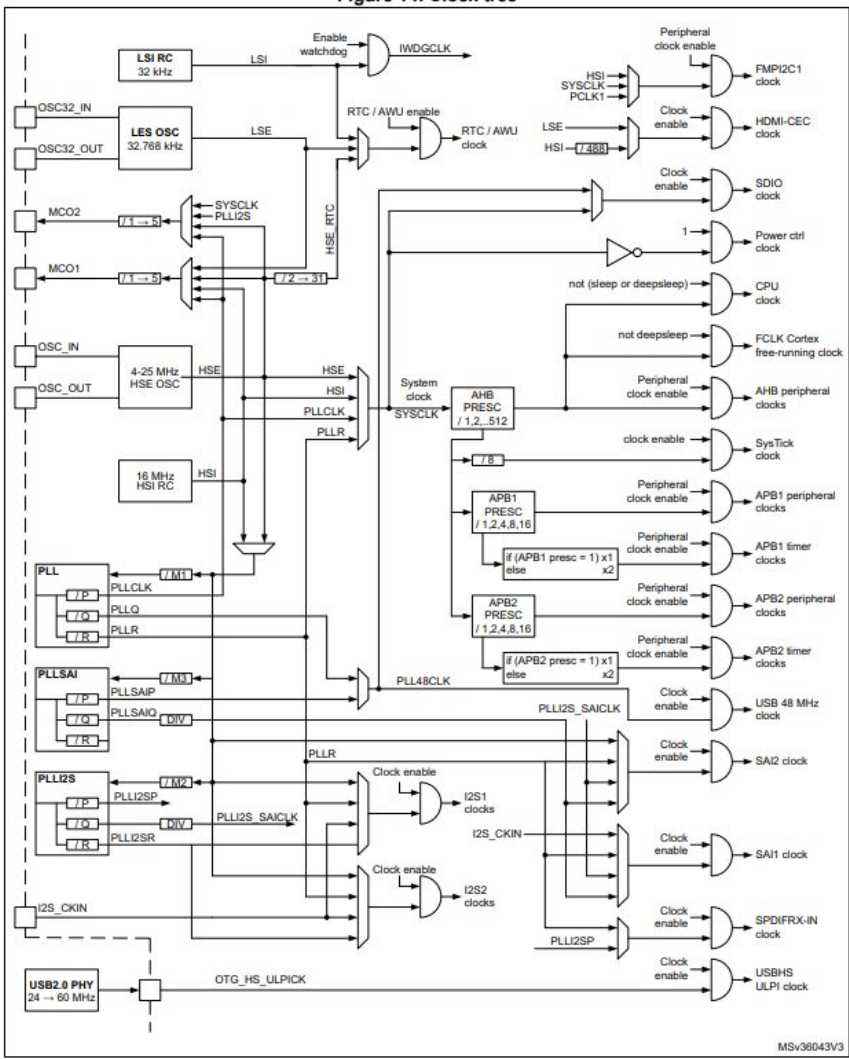
- Ex: The STM32 has a 16MHz clock
  - It can execute 16 million instructions per second

STM32 Clock Tree

[Reference Manual]

Page 118

Figure 14. Clock tree



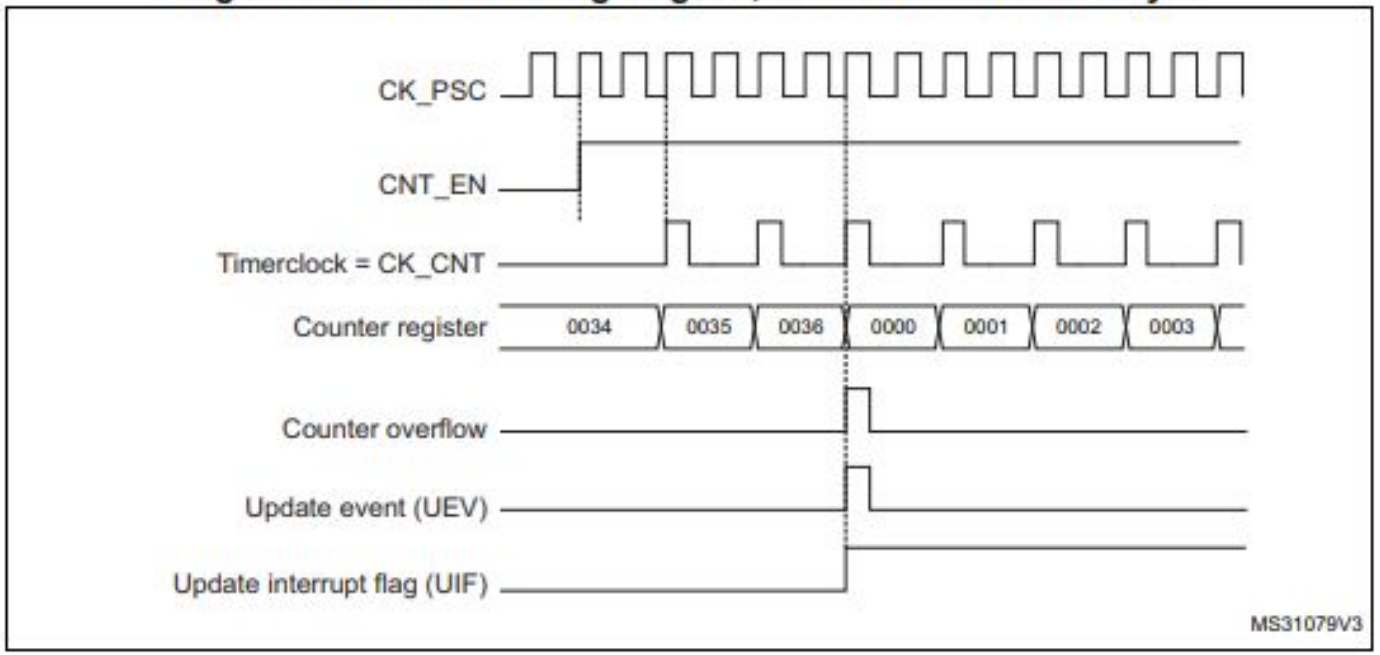
Adding Delays to our Code

```
for(volatile int i = 0; i < “some big number”; i++){
    // Do nothing
}
```

- How long are we waiting?

Timers

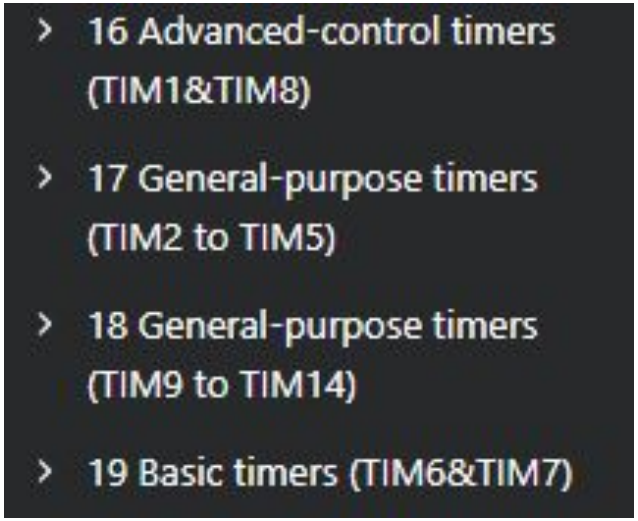
Figure 114. Counter timing diagram, internal clock divided by 2



There are a lot of different timers

- Main differences
  - 16 vs 32 bit
    - Max value of 65,535 or 4 billion
  - PWM

- We’re just going to stick with timer 1



Helpful Registers

16.4.1 TIM1&TIM8 control register 1 (TIMx_CR1)

Address offset: 0x00

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

16.4.5 TIM1&TIM8 status register (TIMx_SR)

Address offset: 0x10

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CC4OF	CC3OF	CC2OF	CC1OF	Res.	BIF	TIF	COMIF	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

- Timer options
  - Turning it on
- Bit 0 tells you if your timer overflowed

More helpful registers

16.4.10 TIM1&TIM8 counter (TIMx_CNT)

Address offset: 0x24

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **CNT[15:0]**: Counter value

16.4.11 TIM1&TIM8 prescaler (TIMx_PSC)

Address offset: 0x28

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 15:0 **PSC[15:0]**: Prescaler value

The counter clock frequency (CK_CNT) is equal to f_{CK_PSC} / (PSC[15:0] + 1).
PSC contains the value to be loaded in the active prescaler register at each update event (including when the counter is cleared through UG bit of TIMx_EGR register or through trigger controller when configured in “reset mode”).

16.4.12 TIM1&TIM8 auto-reload register (TIMx_ARR)

Address offset: 0x2C

Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- Current value of your timer
  - Eg: how long its running
- How often your count increases
- Value your timer counts up to before overflowing

Hands on Time

- The main goal is to Create an Accurate Delay function in TIM1
- There are a couple different ways to do this
    - Configure a timer to overflow every ms then have a for loop count the number of ms
    - Do everything in the timer
        - Configure the prescaler to tick every ms
        - Wait until the counter reach your waiting time
- Don’t Forget
    - Enable Peripheral Clock in RCC

Cheat Sheet

Reference Manual

```
REG |=  (1U << n);    // set bit n

REG &= ~(1U << n);    // clear bit n

REG ^=  (1U << n);    // toggle bit n
```

```
#define GPIOA_Base 0x40020000

#define GPIOA_IDR *((volatile uint32_t*)(GPIOA_Base + 0x10))

#define GPIOA_ODR *((volatile uint32_t*)(GPIOA_Base + 0x14))
```

Anonymous Feedback



Link Here